

51 万行恢复源码 架构与实现深潜

聚焦运行时主链、权限执行与多 Agent 协作三条最关键实现链路。

规模

- 约 1,902 个源码文件
- `.ts/.tsx` 512,664 行

结论

- 不是聊天 CLI
- 是终端 Agent Runtime

Query Loop

Permissions

Agent / Swarm

MCP / Plugins

公众号



先给结论：这套代码的中心不是 UI，而是 runtime orchestration

如果把它看成“带几个工具的终端聊天程序”，后面很多设计都会误判。

判断 1

- 核心对象不是 prompt，而是长期会话与 turn loop。
- `QueryEngine` 和 `query.ts` 明确分离会话层与单轮层。
- 工具执行结果会回流到下一轮推理。

判断 2

- 权限系统不是补丁，而是 tool runtime 的主路径之一。
- hooks、rules、UI prompt、classifier 会共同参与决策。
- 这使 Claude Code 更像受控执行器。

判断 3

- 多 Agent 不是演示特性，而是工程化子系统。
- `AgentTool`、`runAgent`、`inProcessRunner`、swarm 形成完整链路。
- MCP / skills / commands 则是外部能力接入层。

Section

恢复仓库与架构问题

先确认这是什么代码，再谈分析的可信边界。

O1

从恢复方式、规模与目录地形，反推真正的系统边界。

这不是官方 monorepo，而是从 source map 还原出的高保真快照

因此它非常适合做架构与实现分析，但不能假设构建资产、脚手架、测试和内部工具链完整保留。

恢复方式

- ``extract-sources.js`` 读取 ``package/cli.js.map`` 的 ``sources`` 与 ``sourcesContent``。
- 把文件逐个写回 ``restored-src/``，同时做 webpack 路径清洗。
- README 记录恢复文件数为 4,756，来源包版本为 ``2.1.88``。
- 因此分析对象是“编译后快照所映射的源码结构”。

对分析的影响

- 代码主体、注释、目录边界和绝大多数运行逻辑仍然可信。
- 缺失的更可能是构建过程、测试框架、仓库自动化与部分资源文件。
- 对于理解 runtime、tooling、权限和 agent 系统，已足够深。
- 真正重要的是运行时设计，而不是提取脚本本身。

所以我们把它当成“可阅读、可追调用链的运行时快照”。

规模已经指向平台化产品，而不是“小 CLI”

从文件数和目录分布看，UI、运行时、工具与扩展层都很重。

体量

- `src` 下约 1,902 个源码文件
- `.ts/.tsx` 总计 512,664 行
- README 记载恢复文件 4,756 个

高频目录

- `utils` 564
- `components` 389
- `commands` 207
- `tools` 184

意味着什么

- 终端 UI 是大子系统
- Tool/Command 层很厚
- 复杂度重心在 orchestration

目录地形暴露了几组稳定边界

真正该看的不是单个文件，而是这些目录如何形成长期稳定的架构分层。

运行时主干

- ``entrypoints`` / ``main.tsx``: 启动、模式分流、初始化编排。
- ``QueryEngine.ts`` / ``query.ts``: 会话生命周期与单轮递归 loop。
- ``tools`` / ``services/tools``: 模型可用能力与执行框架。
- ``state`` / ``hooks``: React UI 与非 UI 运行逻辑共享状态。

能力与产品层

- ``skills`` / ``plugins`` / ``commands`` / ``services/mcp``: 扩展与集成。
- ``tools/AgentTool`` / ``utils/swarm``: 多 Agent runtime。
- ``ink`` / ``components`` / ``cli/print.ts``: 终端 UI 与 headless 输出。
- ``bridge`` / ``remote``: 不同 surface 共用同一套 runtime。

这些目录不是并列功能点，而是被 Query loop 串起来的多层系统。

官方文档讲的是产品承诺，源码揭示的是实现机制

把 overview 和恢复源码放在一起看，能更快识别哪些说法是真正由架构支撑的。

Overview 层面的说法

- Claude Code 在不同 surfaces 上复用相同 engine。
- 它具备 tools、permissions、MCP、agents 等能力。
- 终端交互、自动化和协作被统一为一套 agent 工作流。
- 文档强调的是体验与概念一致性。

源码层面的证据

- ``entrypoints/cli.tsx`` 先做模式分流，再动态载入 ``main.js``。
- ``cli/print.ts``、``bridge/*``、``QueryEngine.ts`` 都表明多 surface 共用 runtime。
- ``toolExecution.ts`` + ``toolHooks.ts`` 把权限做进了执行主路径。
- ``AgentTool`` / ``runAgent`` / ``inProcessRunner`` 把 agent 真正运行起来。

所以这套系统不是“文档上看起来像 agent”，而是代码上确实按 agent runtime 组织。



总体先抽象为 3 个主层

先用三层建立大图，再在后续章节把每一层拆开。

入口层

- CLI fast-path
- main.tsx startup orchestration
- bridge / remote ingress

内核层

- QueryEngine
- query.ts recursive turn loop
- context / prompt / budget assembly

执行与扩展层

- Tool runtime + permissions
- MCP / skills / plugins / commands
- Agent / swarm / UI surfaces

Section

运行时主链

这部分回答一个关键问题：从用户敲下回车，到 agent 开始递归执行，系统到底怎么串起来。

O2

主链先解释清楚，后面的深潜才不会失焦。

`entrypoints/cli.tsx` 是轻量级分发器，而不是业务中心

它最重要的代码特征是：尽可能晚地加载 `main.js`。

实际职责

- 先处理 `--version` 这类零依赖快路径。
- 把 daemon、bridge、environment-runner 等特殊子命令做早期分流。
- 只有普通交互路径才动态 import `main.js`。
- 注释直接说明这样做是为了减少启动时模块评估开销。

架构意义

- 单二进制承载多个运行模式。
- 启动时延被明确当成产品目标。
- CLI 不只是交互壳，而是 runtime launcher。
- 这也说明很多能力并不依赖终端 UI 本身。

`main.tsx` 是重量级 startup orchestration

如果只想快速建立全局图，这个文件优先级非常高。

它初始化什么

- 配置、鉴权、用户信息、模型与 feature gates。
- MCP、plugins、skills、policy limits 等扩展能力。
- LSP、teleport、render、session restore、analytics。
- 最终把 AppState、UI 和 query runtime 接好。

说明什么

- 这已经是平台产品的初始化复杂度。
- 代码里有大量并发预取、prefetch 和 latency 注释。
- 真正的系统边界在 startup orchestration 就已暴露。
- 很多子系统虽然分文件，但由 `main.tsx` 负责编排。

UI 层和 runtime 通过统一状态与统一输入队列耦合

这一步决定了 REPL、bridge、headless、attachments 最后能否走同一条主链。

状态层

- `AppStateProvider` 暴露 React 订阅接口，同时保留 `getState/setState` 给非 React 代码。
- 权限上下文、task 状态、MCP 状态、voice/mailbox 都能挂到主 store。
- 终端 UI 不是独立状态机，而是共享 runtime 状态的一个 surface。

输入层

- `handlePromptSubmit.ts` 把输入归一成 `QueuedCommand`。
- `processUserInput()` 再把 slash command、bash input、图片等统一编入流程。
- 这样 command surface 和模型 turn loop 才能保持一条主链。

Slash command 与 Bash mode 都是主链入口，不是边角功能

命令面和 shell 面被做成一等入口，说明系统预设用户会显式控制 agent。

Slash command

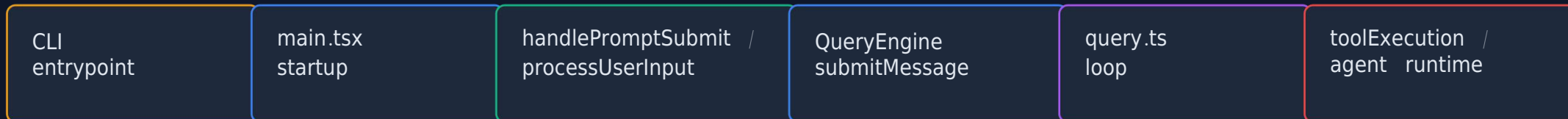
- ``processSlashCommand.tsx`` 解析命令、参数和上下文。
- 命令结果可直接转成 message 序列回注对话。
- 支持 ``context: fork`` 的命令把工作下放给 sub-agent。
- skills 在很多场景下最终会下沉为 command。

Bash mode

- ``processBashCommand.tsx`` 先把原始 shell 输入包装成 ``<bash-input>`` message。
- 同时更新前台 JSX 进度 UI，再继续进入统一消息链。
- shell 不是外挂，而是 Claude Code 的核心工作面之一。
- 这也是 BashTool 设计复杂度很高的原因。

主执行链时序图

这一页用一条链把前面几个文件真正连起来。



链路解释

- 用户输入并不会直接打到模型，而是先进入 `command/input` 归一层。
- ``QueryEngine`` 负责会话级状态，``query.ts`` 负责单轮递归执行。
- `tool runtime` 的输出会被重新拼回消息，形成下一轮递归的输入。
- `bridge`、`headless`、`SDK` 不是重写一套逻辑，而是复用这条主链。

Section

QueryEngine / query.ts 深潜

这部分回答第二个核心问题：Claude Code 的 agent loop 究竟怎么落在代码上。

03

重点不是‘发一次 API 请求’，而是怎样维持一整个会话。



`QueryEngine` 是会话对象，不是 request wrapper

代码注释已经明确写出：one QueryEngine per conversation。

关键字段

- `config`：当前会话的运行时配置。
- `mutableMessages`：整个会话累积消息。
- `abortController`：终止当前 turn 或整个会话。
- `permissionDenials` / `totalUsage` / `readFileState`：权限、成本、文件缓存。

关键职责

- `submitMessage()` 启动新一轮 turn，同时复用会话上下文。
- 把 SDK/headless 场景所需的状态管理从 REPL 抽出来。
- 封装 orphaned permission、skill discovery、session persistence 等会话级行为。
- 最终把实际递归工作下沉给 `query()`。

`submitMessage()` 的真正价值是“包住 query loop 的外层生命周期”

这一层负责的事情很多，但它们都不是 token 级生成逻辑。

外层生命周期动作

- 清理 `discoveredSkillNames`，重设 `cwd`，准备 `session persist`。
- 包装 `canUseTool`，把拒绝事件记录成 SDK 可见的 `permission denial`。
- 构造 `ToolUseContext`，把 `app state`、`commands`、`agents`、`mcp` 等打包进去。
- 迭代 `query()` 的产出，再转换成 SDK `messages`、`usage`、`status` 更新。

关键代码位点

```
class QueryEngine {  
  private mutableMessages: Message[]  
  private permissionDenials: SDKPermissionDenial[]  
  private totalUsage: NonNullableUsage  
  
  async *submitMessage(...) {  
    const wrappedCanUseTool = async (...) => { ... }  
    for await (const message of query({...})) { ... }  
  }  
}
```

为什么要分这一层

- 会话状态与单轮递归状态的职责不同。
- `headless/SDK` 需要稳定 API，但 `REPL` 内部实现可以继续演化。
- 这层把成本统计、权限拒绝记录、`session replay` 这些横切关注点集中起来。

`buildQueryConfig()` 先把 query 级别的不可变配置快照下来

这页讲的是 `query.ts` 开头为什么先构造 config，而不是一路直接读全局状态。

源码里快照了什么

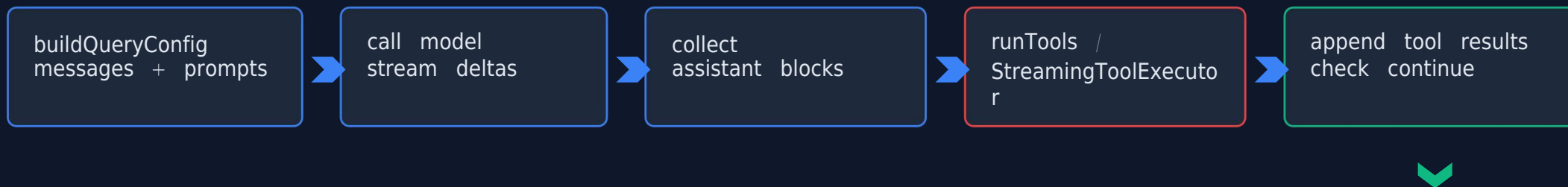
- ``sessionId``。
- ``streamingToolExecution``、``emitToolUseSummaries``、``isAnt``、``fastModeEnabled`` 等 gates。
- 这些值在一次 `query()` 调用内保持稳定，不跟随后续外部状态漂移。
- 注释明确说这是为了未来把 step 提纯成 `(state, event, config)`` reducer。

设计意义

- 减少 query loop 中途读取全局状态带来的漂移问题。
- 把 tree-shaking 边界与真正的运行时配置边界分开。
- 说明作者在为更纯的 step/state machine 结构预留演化空间。

`query.ts` 才是 agent loop 的递归核心

这一页不是概念图，而是对应源码里的真实循环骨架。



源码信号

- ``toolUseBlocks`` 是循环是否继续的关键退出信号。
- `stop_reason=`tool_use`` 不可靠，因此流式过程中显式收集 block。
- Tool results 被编码成新的 `user/attachment messages` 回注。
- `needsFollowUp` / `maxTurns` / `stop hooks` 共同控制递归边界。

设计含义

- 模型输出不是终点，而是下一步执行计划。
- 工具结果不是副作用，而是下一轮推理输入。
- 这就是 Claude Code 作为 agent runtime 的真正核心。

流式阶段不是简单吐字，而是在构建可执行结构

`query.ts` 里真正重要的是 `assistant stream` 如何变成 `tool-use-aware` 的结构化消息。

流式处理做了什么

- 增量消费 `stream\_event`，分别更新 UI、统计和 `assistant blocks`。
- 显式维护 `assistantMessages`、`toolResults`、`toolUseBlocks`。
- 如果 `feature gate` 打开，会启用 `StreamingToolExecutor`。
- 一旦检测到 `tool use`，就不是“结束输出”，而是“切入执行阶段”。

关键变量

```
const assistantMessages = []
const toolResults = []
const toolUseBlocks = []
let needsFollowUp = false

let streamingToolExecutor =
  useStreamingToolExecution
    ? new StreamingToolExecutor(...)
    : null
```

为什么要这么设计

- UI 流、tool 流、usage 流并行存在。
- 这样既能保留实时交互，又不会失去 `agent loop` 的控制权。
- 这也是 `query.ts` 复杂度远高于普通 `SDK wrapper` 的原因。

`query.ts` 还处理 fallback、withheld error 和 orphan 清理

这部分不是锦上添花，而是长时任务稳定性的关键。

流式恢复机制

- prompt-too-long、media-size、max-output-tokens 这类错误会先被 withheld，等 recovery 逻辑判定是否能自救。
- 如果发生 streaming fallback，会给旧 assistant message 发 tombstone，并清空旧的 toolUse / toolResult 暂存。
- 随后重建 `StreamingToolExecutor`，避免旧 tool_use_id 泄漏进 retry。

源码级含义

- runtime 在显式处理“不完整 thinking / 不完整 tool use / fallback 重试”这些脏状态。
- 这也是 ``yieldMissingToolResultBlocks()``、``stripSignatureBlocks()``、fallback warning message 会出现的背景。
- `query.ts` 的职责并不是“问模型”，而是“维护一个可持续迭代的执行回路”。

`query.ts` 还负责预算、续跑、压缩与回退模型

单轮循环并不只是‘执行工具’，它还承担大量运行时治理职责。

治理机制

- `taskBudget` 用于整个 agentic turn 的 API 预算控制。
- current turn token budget / output tokens 会动态计算。
- 遇到上下文膨胀时，会触发 compact / microcompact / snip replay。
- 必要时会使用 fallback model 或 continuation 计数。

对产品的意义

- 系统不是把模型当无限资源使用。
- 长任务需要 runtime 主动维持上下文可持续性。
- 这让 coding session 能在真实大工程里跑得更久。
- 也是它能承担 agent task 的基础设施条件。

`QueryEngine` 与 `query.ts` 的分工很清晰

这一点值得特别讲，因为它是整个系统可演进性的关键。

QueryEngine 负责

- 会话级状态与 SDK/headless 接口。
- 权限拒绝记录、usage 汇总、message replay。
- session persistence、orphaned permission、skill discovery 等外围生命周期。

`query.ts` 负责

- 当前 turn 的 prompt / context / config 组装。
- 模型流式调用、tool use 收集、工具执行、继续或结束。
- 预算与 compact 决策，直到本轮稳定收敛。

这种拆法让“会话对象”和“单轮状态机”都能保持独立复杂度。

Section

Tool Execution / Permissions 深潜

这一章解释 Claude Code 最像“受控执行器”的地方：模型提出操作，runtime 决定能不能执行、怎么执行。

04

权限不是外围 UI，而是工具执行主路径的一部分。

Tool 抽象把能力、描述、权限与调用统一起来

模型看到的是 tool schema , 但 runtime 看到的是更重的执行对象。

Tool 对象包含

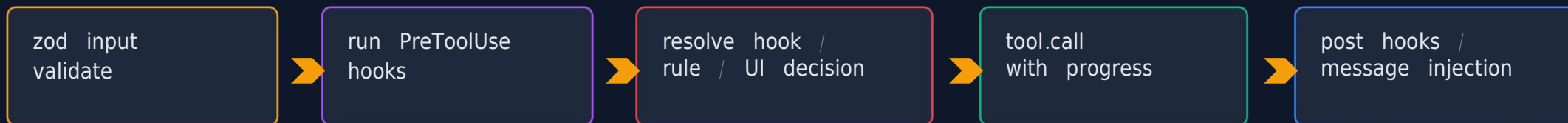
- 名字、输入 schema、描述函数、call 实现。
- `requiresUserInteraction` 等权限相关信号。
- tool permission context 与 agent/task 相关上下文。
- progress 事件与 hook 交互能力。

Registry 侧做的事

- `tools.ts` 汇总 built-in tools、MCP tools 和 agent tools。
- 最终暴露给模型的是当前 permission mode 下可见的 tool pool。
- Tool 不只是函数，而是可被治理的能力单元。

`toolExecution.ts` 执行流程图

这一页对应的是真正跑一次 tool use 时的主路径。



源码关键信号

- `streamedCheckPermissionsAndCallTool()` 把进度与最终结果并到一个 async iterable。
- `checkPermissionsAndCallTool()` 里先做 schema 校验，再进入 hook + permission 流。
- tool span / analytics / tracing 都在这条主路径上埋点。

这意味着

- 工具执行不是黑盒函数调用，而是一条治理链。
- 权限、hooks、用户修改、analytics 都会改变最终输入与行为。
- 这也是为什么 tool runtime 能支撑真正的 coding agent。

`runPreToolUseHooks()` 的返回类型，本身就是一套控制协议

如果只把 hook 当成“执行一个脚本”，就会低估它在主链里的地位。

它能返回什么

- ``message``: 把 hook 进度或附件直接注入 UI / transcript。
- ``hookPermissionResult``: 直接给出 allow / ask / deny。
- ``hookUpdatedInput``: 只改输入，不改权限决议。
- ``preventContinuation``、``stopReason``、``additionalContext``、``stop``: 改写后续控制流。

这意味着

- Pre hook 不只是策略判断器，还是消息注入器和控制流改变器。
- tool runtime 因此天然支持企业策略、包装式交互、动态附加上下文。
- 真正排查某个 tool 为什么没按预期执行，必须先看这里。

PreToolUse hooks 会在权限判断前先介入

这使工具执行支持外部策略、输入修正、拦截与摘要化说明。

主路径细节

- ``runPreToolUseHooks()`` 会返回 `stop/info/updatedInput` 等结果。
- `hook` 既可能直接 `deny`，也可能 `allow` 并注入 ``updatedInput``。
- 工具执行层还会统计 `hook` 数量和耗时，并在特定用户类型下显示摘要。
- 因此 `hook` 不是旁路机制，而是第一层策略注入点。

位点

```
for await (const result of runPreToolUseHooks(...)) {  
  if (result.type === 'stop') ...  
  if (result.updatedInput) processedInput = ...  
}
```

设计后果

- 权限结果不再只是 UI prompt 的返回值。
- 外部规则、headless wrapper、企业策略都能在这里改变行为。
- 也因此调试工具执行时，必须先看 `hooks` 再看 ``tool.call()``。

resolveHookPermissionDecision() 暴露了权限优先级

这是权限系统最值得读的函数之一。

它的决策顺序

- 如果 hook `allow`，先看这个工具是否仍要求交互，或当前上下文是否强制 `canUseTool`。
- 若无需再次交互，再跑 `checkRuleBasedPermissions()`，deny/ask 规则仍可覆盖 hook allow。
- 如果 hook `deny`，直接拒绝。
- 如果没有 hook 决策或 hook=`ask`，再进入常规 `canUseTool()` 流程。

位点

```
if (hookPermissionResult?.behavior === 'allow') {  
  const ruleCheck = await checkRuleBasedPermissions(...)  
  if (ruleCheck === null) return hookPermissionResult  
  if (ruleCheck.behavior === 'deny') return ruleCheck  
  return await canUseTool(...)  
}
```

为什么重要

- 它说明 hook 并不能完全跳过规则系统。
- 权限不是单点裁决，而是 hook、规则、交互 prompt 三层叠加。
- 这也是系统能同时支持本地规则、企业策略和用户交互的原因。

`runPostToolUseHooks()` / failure hooks 负责把 执行结果再加工一次

也就是说, `tool.call()` 返回后, 结果还可能被 hook 继续改写、阻断或补充。

成功路径上的 post hook

- 可以产出额外 attachment/message。
- 可以 `preventContinuation`, 直接阻止 query loop 继续向前。
- 对 MCP tool 还可以 `updatedMCPToolOutput`, 即直接改写输出。
- 出现 `blockingError` / `execution error` 时, 也会转成 hook 级 attachment。

失败路径上的 failure hook

- `runPostToolUseFailureHooks()` 会拿到 `error` 与 `isInterrupt`。
- 因此失败也能被策略化处理, 而不只是抛错结束。
- 从 runtime 角度看, 工具执行的成功与失败都被统一纳入 hook 协议。

Permission mode 是策略切换器，而不是单个 bool

`PermissionMode.ts` 把不同模式映射到外部可见语义与 UI 表示。

源码里定义的模式

- `default`、`plan`、`acceptEdits`、`bypassPermissions`、`dontAsk`。
- 在特定 feature 下还有 `auto`。
- 每个 mode 都有 title、shortTitle、symbol、color、external 映射。
- 所以 permission mode 同时服务于策略、UI 与对外接口。

规则与文件系统

- 规则字符串通过 `permissionRuleParser.ts` 解析，例如 `Bash(npm install)`。
- 文件系统权限检查在 `utils/permissions/filesystem.ts` 中深入处理 working dir、读写范围与建议。
- shell 规则匹配则落在 `shellRuleMatching.ts`。
- 权限本质上是 mode + rules + path/shell analysis 的组合。

真正的 `tool.call()` 发生在权限决议之后，但事情还没有结束

Claude Code 还会继续把结果变成可追踪、可后处理、可回注的消息。

call 前后发生什么

- 执行前会回填/修正 `callInput`，尤其是文件路径类输入。
- `tool.call()` 得到 `progress` 回调，继续转成 SDK/UI 可见事件。
- 执行耗时会进入 `tracing` 与 `usage` 统计。
- 用户修改输入与 `classifier` 决策也会被带入上下文。

结果如何回到主链

- 权限拒绝时会生成 `stop message` 和 `denial hooks`。
- 成功时会拼装 `tool result message`，继续送回 `query loop`。
- 随后还会运行 `post-tool hooks` / `failure hooks`。
- 因此工具执行结束不等于一次 `turn` 结束。

Section

AgentTool / Swarm 深潜

这一章解释多 Agent 为什么不是装饰功能，而是 runtime 的自然延伸。

05

主 agent 只是第一个 agent；系统从一开始就允许它再生成更多 agent。



`AgentTool` 是模型可见的“创建子代理”接口

在 `schema` 层它像普通 `tool`，在 `runtime` 层它会生成一个新的 `query loop`。

call 参数暴露的能力

- ``prompt``、``subagent_type``、``description``、``run_in_background``、``mode``、``isolation``、``cwd``。
- 既支持单个子代理，也支持 ``team_name + name`` 触发多 agent teammate spawn。
- 会根据 `coordinator mode`、`permission mode`、`agent filtering` 调整行为。

这说明什么

- 子代理不是 UI 层按钮，而是模型可直接调用的工具。
- 所以 `multi-agent` 是 `query loop` 的内生能力，而不是外部 `orchestration shell`。
- 这也是后面 ``runAgent()`` 设计成通用生成器的背景。

`runAgent()` 会重新构造一条子代理 query loop

这个函数本质上就是“生成子代理运行时”的工厂。

它先做什么

- 根据父级 permission mode 与 override 计算 agent model。
- 创建 agentId，必要时设置 transcript 子目录与 Perfetto trace hierarchy。
- 构造 `initialMessages`，并对 forked context 过滤 incomplete tool calls。
- 根据 agent 类型裁剪 `claudeMd`、`gitStatus` 等上下文，降低 token 开销。

位点

```
export async function* runAgent({...}) {  
  const resolvedAgentModel = getAgentModel(...)  
  const initialMessages = [...contextMessages, ...promptMessages]  
  const [baseUserContext, baseSystemContext] = await Promise.all([...])  
  for await (const message of query({...})) { ... }  
}
```

关键判断

- 子代理并不是沿用父代理的所有上下文和规则。
- 系统会基于 agent type 做瘦身、替换、隔离与 transcript 分组。
- 这是规模化使用子代理时必须做的 token 与治理优化。

built-in agents 不是死名单，而是 feature-gated 的 agent registry

Explore、Plan、Guide、Verification 是否出现，都和入口、环境、实验开关有关。

builtInAgents.ts 暴露的事实

- 非交互 SDK 场景可通过环境变量禁用所有 built-in agents。
- coordinator mode 打开时，会转向 `coordinator/workerAgent.js` 提供 agent 列表。
- Explore/Plan/Verification 都受 feature 与 GrowthBook 开关控制。
- Code Guide agent 只在非 SDK entrypoint 下加入。

架构含义

- built-in agents 是产品策略层，不是硬编码常量表。
- 它们跟入口模式、用户类型、实验流量和权限模式深度耦合。
- 这也解释了为何 agent runtime 要独立成体系，而不是塞进 UI 逻辑里。

inProcessRunner.ts` 证明 swarm 不是口号，而是真实运行时

它把 `runAgent()` 包成 in-process teammate runner，并补上共享终端所需的桥接逻辑。

额外承担的职责

- 通过 AsyncLocalStorage/teammate context 做上下文隔离。
- 把进度更新和 AppState 写回 leader 可见状态。
- 在完成后给 leader 发 idle notification。
- 支持 plan mode approval flow、abort cleanup 与 mailbox permission response。

源码开头就写明了

In-process teammate runner

- context isolation
- progress tracking and AppState updates
- idle notification to leader
- plan mode approval flow support
- cleanup on completion or abort

重点机制

- `createInProcessCanUseTool()` 会优先尝试 leader 的 ToolUseConfirm bridge。
- bridge 不可用时，再退化到 mailbox permission sync。
- 这意味着 subagent 也能共享主界面的权限交互，而不是自己绕开。

`createInProcessCanUseTool()` 是 teammate 权限桥

这个函数把 worker 的 ask 权限请求转接回 leader 的 UI 或 mailbox。

标准路径 : leader UI bridge

- 先跑 `hasPermissionsToUseTool()` , allow/deny 直接透传。
- 若是 bash 且 classifier 可自动批准, 会先等待 classifier 结果。
- 仍需 ask 时, 优先把请求塞进 leader 的 `ToolUseConfirm` queue。
- 允许时还会把 permission updates 写回 leader 的共享 context , 并 `preserveMode` 。

退化路径 : mailbox sync

- bridge 不可用时, 会创建 permission request , 发到 leader mailbox。
- worker 端轮询自己的 mailbox , 等待 approved/rejected 响应。
- 因此 swarm 的权限交互即使脱离共享终端, 也不会失效。

swarm 的运行机制建立在 mailbox、task 与 permission sync 上

也就是说，团队协作并不是神秘黑盒，而是明确的数据交换与状态同步协议。

运行机制

- leader / teammate 之间通过 mailbox 交换消息与状态。
- task 系统负责 claim、list、update 和输出持久化。
- permissionSync 把权限请求和响应同步到团队上下文。
- 必要时还会持久化 transcript、permission update 与 task output。

为什么这很重要

- 说明多 Agent 已经是可恢复、可追踪、可治理的子系统。
- 不是简单并发调用模型，而是带状态的一组协作实体。
- 这也是为什么 swarm 目录下会有一整套 helpers、locks、mailbox 与 bridge。

waitForNextPromptOrShutdown() 把 teammate 变成持续存活的协作实体

这也是 in-process teammate 和一次性 background task 最大的差异。

轮询逻辑

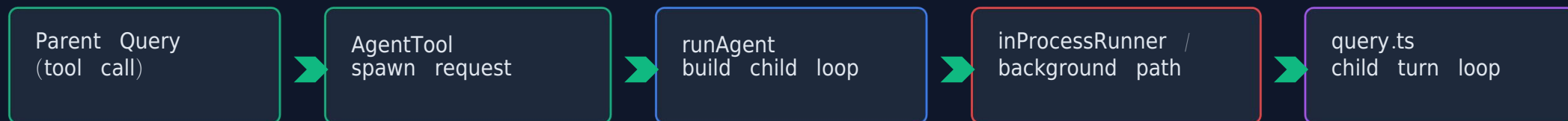
- 每 500ms 检查内存里的 pending user messages。
- 随后查 mailbox，优先处理 shutdown request，再处理 team lead 消息。
- 如果没有消息，还会尝试 claim 团队任务列表里的可领任务。
- 整个循环只有在 abort 或模型批准 shutdown 后才退出。

架构含义

- teammate 不是一次性函数调用，而是持续存活、可接收多轮 prompt 的 actor。
- mailbox + task list + idle notification 一起组成了 swarm 的最小协作协议。
- 这也是源码里会出现 leader / teammate / task owner 这些显式身份的原因。

Agent runtime 关系图

这一页把 AgentTool 之后到底发生什么画成一张图。



共享部分

- 共享 query loop、tool runtime、usage 统计、session 构造方式。
- 子代理照样会进入 permissions、hooks 与 tracing。
- 因此 child agent 不是特判执行器，而是复用主内核。

特有部分

- 额外的 agent id、transcript、worktree、teammate context。
- leader bridge / mailbox / permission sync。
- built-in agent registry 与 feature-gated spawn policy。

Section

扩展层与多 Surface

最后回到外围系统：能力从哪里来，为什么
terminal / bridge / SDK 可以共用同一套引擎。

06

这部分让架构闭环：内核之外的扩展与输出
面如何接上来。



MCP 是外部能力接入层，而不是内部工具系统的替代品

从源码看，它被做成‘把外部 server 映射成 runtime 可用能力’的协议层。

MCP 客户端侧职责

- 建立 server 连接，管理缓存、配置和资源发现。
- 把外部 tools / commands / resources 映射进本地 runtime。
- 处理 elicitation、错误上报和 logging-safe metadata。
- 最终仍要进入本地的 Tool/Command surface。

架构边界

- MCP 不是 query loop，本质是 external capability bridge。
- 它和 skills/plugins/commands 是互补关系，不是同义词。
- 外部能力进来后，仍要接受本地 permissions 与 tool runtime 治理。

Skill、Plugin、Command 三层必须分开看

这三层经常被混淆，但在源码里边界很清楚。

三层区别

- Plugin 管安装、来源、缓存、manifest 和分发。
- Skill 管 workflow 模板与经验封装，经常最终转成 command。
- Command 是用户显式入口，也是 slash command 的执行对象。
- MCP 则是另一条把外部能力拉进来的协议通道。

它们最后如何汇合

- 模型真正可见的是 tool pool 和 command surface。
- 用户真正操作的是命令、UI 交互和 agent runtime。
- 所以这些扩展层最终都要折叠到共享 runtime 中。



终端引擎 UI、headless/SDK、bridge/remote 共享一套引擎

这正是官方 overview 里“same engine across surfaces”的源码落点。

本地 surface

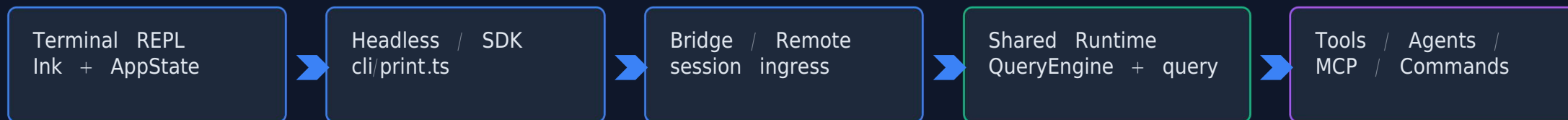
- ``ink/components/App.tsx`` 与 `AppState` 负责终端交互。
- ``cli/print.ts`` 负责 headless/SDK 的 structured output。
- 它们都不是独立业务实现，而是同一 runtime 的不同展示面。

远程 surface

- ``bridgeMain.ts``、``initReplBridge.ts``、``bridgeUI.ts`` 把远端消息重新注入本地队列。
- remote/bridge 共享 `QueryEngine`、`query.ts` 与 `tools/agents`。
- 所以不同入口之间最本质的差异是 ingress/egress，不是推理内核。

多 Surface 关系图

从产品形态回看，真正共享的是中间 runtime，而不是某一个 UI 组件。



对外观测到的差异

- terminal 更强调交互 UI 与权限弹窗。
- headless/SDK 更强调结构化输出与可集成性。
- bridge/remote 更强调消息 ingress 与会话同步。

对源码真正重要的共性

- 都复用 QueryEngine / query.ts / tool runtime / agent runtime。
- 差异主要发生在入口分流和输出通道。
- 这也是整体架构能够长期演进的原因。

从工程风格上看，这套代码有 3 个鲜明特征

这些特征可以帮助你快速判断它和一般 AI 工具项目的区别。

1. 强 runtime 心智

- 会话对象和单轮循环分开
- 权限、budget、compaction 都在主路径
- 大量异步生成器与事件流

2. 强产品工程化

- 终端 UI、bridge、remote 共存
- feature gates 与实验体系深入代码
- MCP / plugin / skill / command 边界明确

3. 强治理能力

- permissions/hook/rules/classifier 叠加
- subagent 也受统一治理
- session/task/transcript 可追踪

如果继续读源码，建议按这条路径推进

这样可以先建立全局图，再逐步下钻，不会在细节里迷失。

第一轮：建模系统

- 先读 ``entrypoints/cli.tsx``、``main.tsx``。
- 再读 ``QueryEngine.ts`` 与 ``query.ts``，建立主链模型。
- 补看 ``Tool.ts``、``tools.ts`` 理解 tool surface。
- 到这里你已经能解释 70% 的系统行为。

第二轮：攻克复杂度

- 读 ``services/tools/toolExecution.ts`` + ``toolHooks.ts``。
- 再读 ``tools/AgentTool/*``、``utils/swarm/*``。
- 最后看 ``services/mcp``、``skills``、``plugins`` 与 ``cli/print.ts``。
- 这条顺序能最大化减少“只见树木不见森林”的问题。

最后判断

一句话总结：Claude Code 的核心创新不是“会调工具”，而是把终端工作流、权限治理、扩展层和多 Agent 协作统一到一个 runtime 里。

架构结论

- 这是终端 Agent Runtime，不是聊天 CLI。
- 核心是 Query loop、Tool runtime、Permission system 与 Agent runtime。
- MCP / skills / plugins / commands 是围绕这个内核组织的扩展层。

对 AI agent 设计的启发

- 真正可用的 agent 系统必须把 permissions/budget/replay 做进主链。
- 多 agent 要可追踪、可恢复、可治理，不能只是并发调用模型。
- 不同 surfaces 共用同一引擎，才能让产品与平台同时成立。

关键源码文件索引

这页可以直接当作二次阅读导航。

主链

- `entrypoints/cli.tsx`
- `main.tsx`
- `QueryEngine.ts`
- `query.ts`

执行

- `Tool.ts`
- `tools.ts`
- `services/tools/toolExecution.ts`
- `services/tools/toolHooks.ts`

多 Agent / 扩展

- `tools/AgentTool/\*\*`
- `utils/swarm/\*\*`
- `services/mcp/\*\*`
- `skills/\*\*`, `plugins/\*\*`

用这 3 组词汇记住整套系统

这也是给听众做信息压缩的最好方式。

Conversation vs Turn

- `QueryEngine` = conversation owner
- `query.ts` = turn loop
- 两层职责不要混

Capability vs Control

- Tool/MCP/Plugin 提供能力
- Permission/Hook/Budget 提供控制
- 两者都在主链里

Parent vs Child Agent

- `AgentTool` 触发生成
- `runAgent` 构造 child runtime
- `swarm` 负责协作机制

公众号

把二维码放在结尾页，便于你直接拿去汇报或对外分享。

后续交流

- 如果继续扩展这份 deck，可以把 `query.ts`、`toolHooks.ts`、`inProcessRunner.ts` 单独拆成分享。
- 也可以继续做“Claude Code 与 Codex / Cursor / Cline”的实现对比版。
- 这页保留二维码，适合直接作为结尾页。

扫码关注



Analysed by Phodal with Codex & ChatGPT Slide skill